रा.इ.सू.प्रो.सं
NIELIT
GUWAHATI

National Institute of Electronics and Information Technology
Ministry of Electronics and Information Technology
Government of India

DAY 2

Tidyverse Data Manipulation in R

A practical, dataset-driven approach for biological data science

filter() • select() • mutate() • arrange() • group_by() • summarise() • pipes

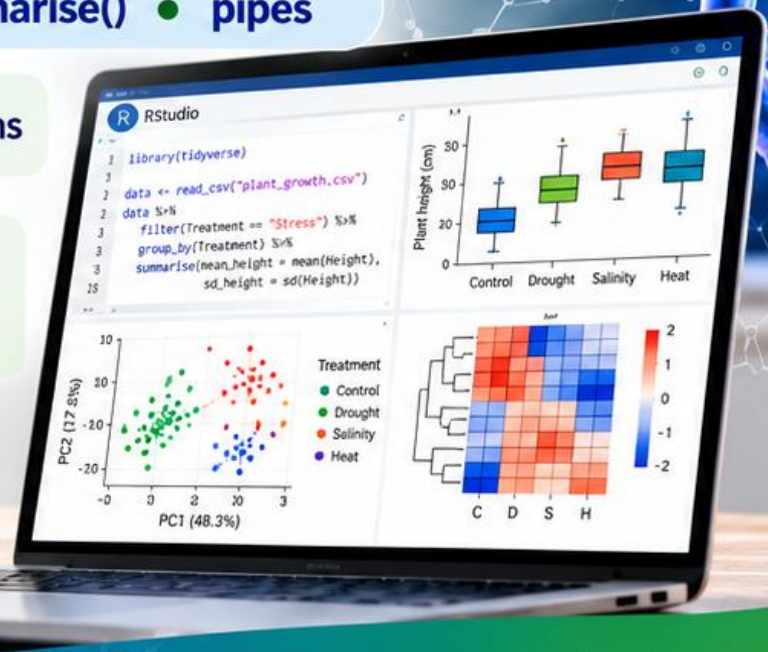
Example application: Plant-growth analysis under control and stress conditions

Course: R for Biological Data Science

Duration: 1.5 Hours

Level: Beginner





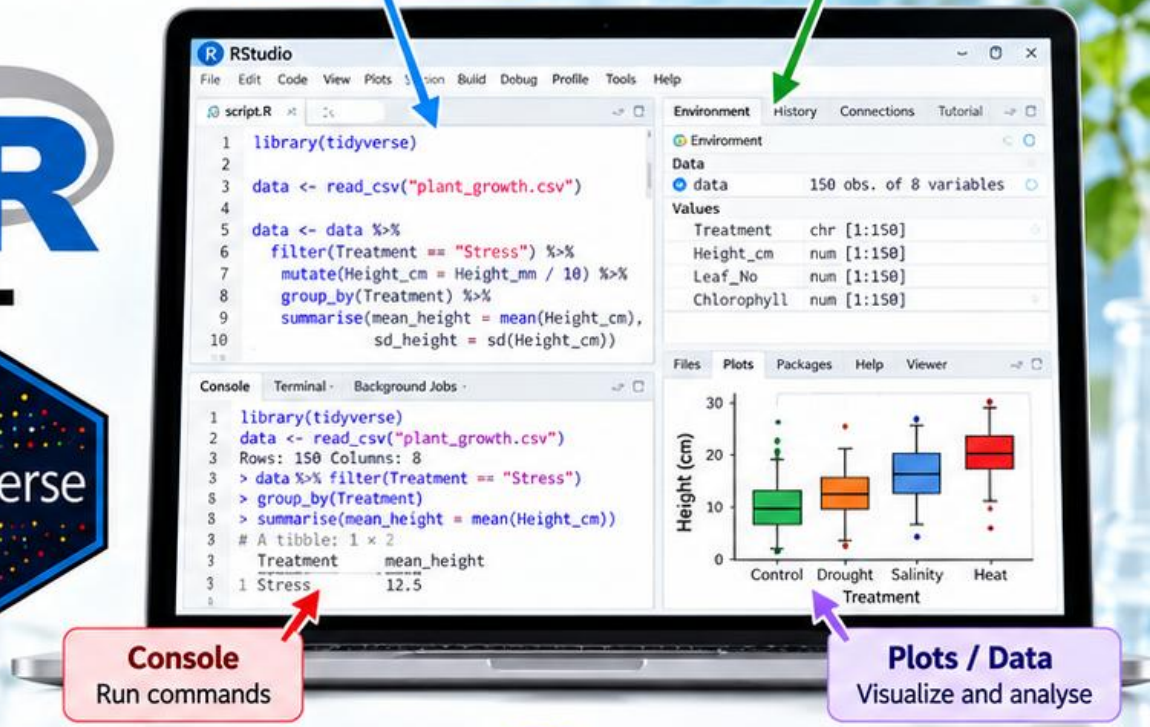
R tidyverse

By
Manabendra Nath
Senior Bioinformatics Faculty, NIELIT Guwahati

Tidyverse Data Manipulation in R

1. Understand **tidyverse data manipulation.**
2. Import and inspect a **biological dataset.**
3. **filter()** and **select()** observations and variables.
4. **mutate()** and **arrange()** data.
5. **group_by()** and **summarise()** biological measurements.
6. Use the **pipe %>%** to build **reproducible workflows.**









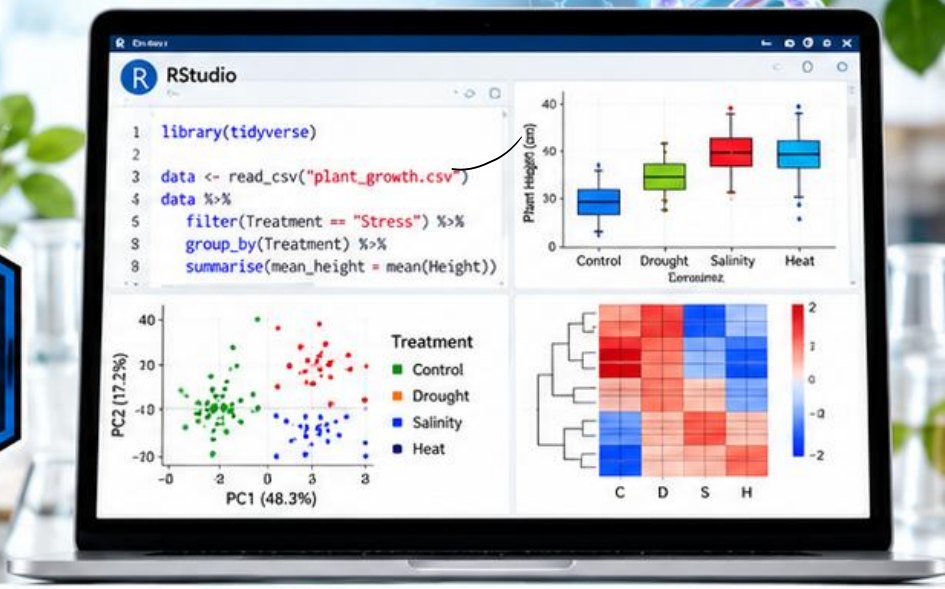
Introduction to Tidyverse

DAY 1

R & RStudio Fundamentals

From the History of R to Basic R Programming for Biological Data Science

The **tidyverse** is a collection of **R packages** designed for data science. It provides a **consistent framework** for **importing, cleaning, transforming, summarising,** and **visualising** data.



Key tidyverse Packages

dplyr
Data manipulation (filter, select, mutate, arrange, summarise, etc.)

tidyr
Data organisation and reshaping (pivot, separate, unite, etc.)

readr
Importing rectangular text data (CSV, TSV, etc.)

ggplot2
Data visualisation (publication-quality plots)

tibble
Modern data frames (easy to use, efficient)

stringr
Text manipulation (pattern matching, string operations)

Important Features

- Consistent syntax across packages
- Works well with the pipe (%>%)
- Easy to learn and readable code
- Designed for reproducible research
- Integrates data manipulation, visualisation and modelling
- Widely used in biological and genomics research

Biological Applications

- **Processing biochemical measurements** (e.g., chlorophyll, proline, enzyme activity)
- **Analysis of plant phenotypic/trait data** (e.g., growth, germination, stress responses)
- **Handling sample metadata** (e.g., treatment, tissue, time-point)
- **Working with gene-expression tables** (e.g., RNA-Seq, DEGs)

Typical Tidyverse Workflow for Biological Data

Import (readr) → Clean (dplyr, tidyr) → Transform (dplyr, tidyr) → Summarise (dplyr) → Visualise (ggplot2) → Report (tibble)

Example Dataset: Plant-Growth Experiment

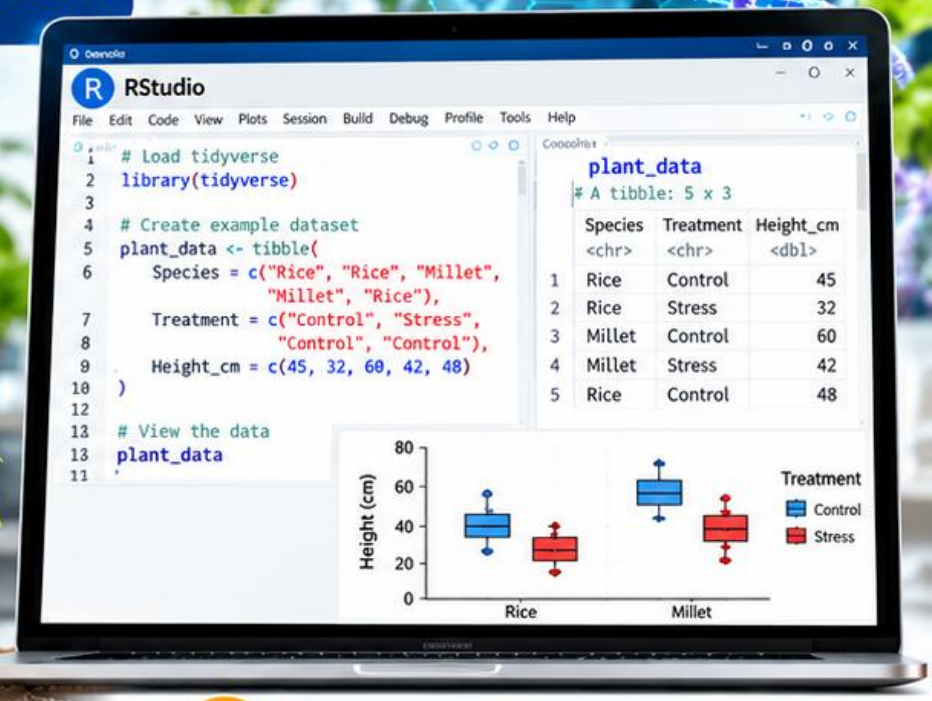
Study Scenario

Consider an illustrative experiment comparing plant height in **Rice** and **Millet** under **Control** and **Stress** treatments.

Example Dataset

	Species	Treatment	Height_cm
1	Rice	Control	45
2	Rice	Stress	32
3	Millet	Control	60
4	Millet	Stress	42
5	Rice	Control	48





Dataset Structure

- **Rows** represent individual observations (each plant/sample).
- **Columns** represent variables (features measured or recorded).
- **Species** and **Treatment** are categorical (factor) variables.
- **Height_cm** is a numerical (continuous) variable.
- Each row corresponds to one biological sample/plant.
- This dataset is small and illustrative; real datasets may contain many more samples and additional variables (e.g., time point, tissue, replicate, etc.).

Variable Types in R

Variable	Type in R	Example Values	Description
Species	**factor** (categorical)	Rice, Millet	Plant species (e.g., rice, millet)
Treatment	**factor** (categorical)	Control, Stress	Experimental treatment
Height_cm	**numeric** (continuous)	45, 32, 60, 42, 48	Plant height in centimetres

Important Points

1. This is a small, illustrative dataset to demonstrate R functions.
2. Real datasets may contain many more samples, variables and potential missing values.
3. Species and Treatment are categorical variables (factors) and can be used for grouping and comparison.
4. **Height_cm** is a numerical variable suitable for summary statistics and visualisation.
5. We will use this dataset to demonstrate importing, inspecting, filtering, summarising and visualising using **tidyverse**.

5 Loading Tidyverse and Creating the Dataset

1 Install and Load Package

```r
# Install tidyverse (run only once)
install.packages("tidyverse")

# Load tidyverse
library(tidyverse)
```

Run once (only for first time)

Load package

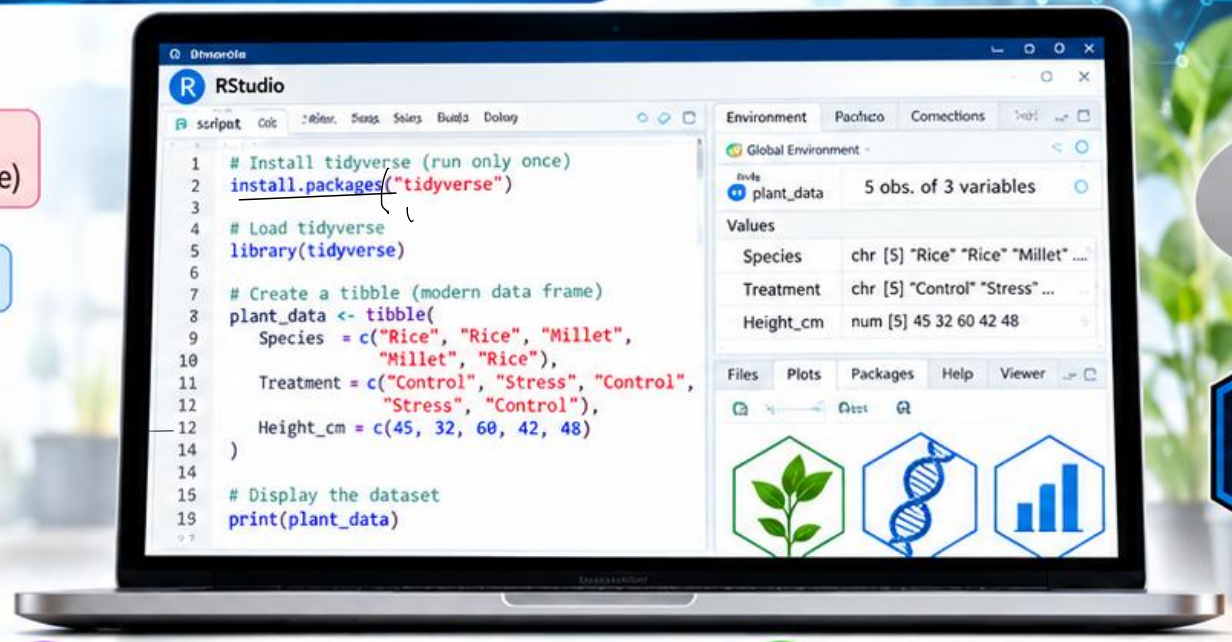
2 Create the Example Dataset

```r
# Create a tibble (modern data frame)
plant_data <- tibble(
    Species   = c("Rice", "Rice", "Millet",
                  "Millet", "Rice"),
    Treatment = c("Control", "Stress", "Control",
                  "Stress", "Control"),
    Height_cm = c(45, 32, 60, 42, 48)
)
```

3 Display the Dataset

```r
print(plant_data)
```





Output in R Console

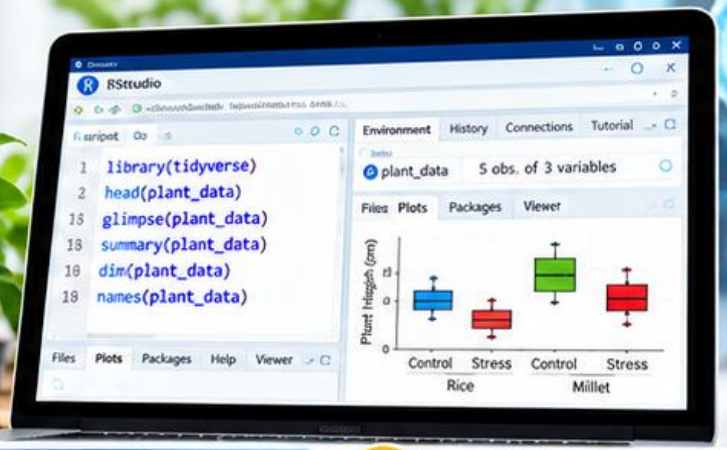
```
> print(plant_data)
# A tibble: 5 × 3
  Species   Treatment Height_cm
  <chr>     <chr>        <dbl>
1 Rice      Control         45
2 Rice      Stress          32
3 Millet    Control         60
4 Millet    Stress          42
5 Rice      Control         48
```

Explanation

1. **library(tidyverse)** loads the tidyverse packages (dplyr, tidyr, ggplot2, readr, etc.).
2. **tibble()** creates a modern data-frame structure that is easy to use and well-formatted.
3. **plant_data** stores the example plant-growth experiment dataset.
4. **print()** displays the dataset for inspection in the console.

6 Inspecting Biological Data

Before manipulating a dataset, inspect its **structure, variable types, and value distributions.** This helps to understand the data, detect possible issues, and ensure correct analysis.



1 head() – View first observations

```
1  head(plant_data)
```

```
# A tibble: 5 × 3
  Species  Treatment  Height_cm
  <chr>    <chr>      <dbl>
1 Rice     Control    45
2 Rice     Stress     32
3 Millet   Control    60
4 Millet   Stress     42
5 Rice     Control    48
```

2 glimpse() – Structure and types

```
1  glimpse(plant_data)
```

```
Rows: 5
Columns: 3

$ Species    <chr> Rice, Rice, Millet, …
$ Treatment  <chr> Control, Stress, …
$ Height_cm  <dbl> 45, 32, 60, 42, 48
```

3 summary() – Descriptive summary

```
1  summary(plant_data)
```

```
   Species           Treatment          Height_cm
Length:5           Length:5           Min.   :32.00
Class :character   Class :character   1st Qu.:42.00
Mode  :character   Mode  :character   Median :45.00
Rice  :3           Control:3          Mean   :45.40
Millet:2           Stress :2          3rd Qu.:48.00
                                      Max.   :60.00
```

Function Interpretation

1. **head()** – displays the first observations.
2. **glimpse()** – shows variable names and data types.
3. **summary()** – provides descriptive summaries.
4. **dim()** – returns the number of rows and columns.
5. **names()** – identifies the variables.

4 dim() – Number of rows and columns

```
1  dim(plant_data)
```

```
[1] 5 3
```

Number of rows (observations)

Number of columns (variables)

5 names() – Variable names

```
1  names(plant_data)
```

```
[1] "Species"   "Treatment"   "Height_cm"
```

Variable Types in Our Dataset

Variable	Type in R	Description
Species	**character** (categorical)	Plant species (e.g., Rice, Millet)
Treatment	**character** (categorical)	Experimental treatment (e.g., Control, Stress)
Height_cm	**numeric** (continuous)	Plant height in centimetres

Best Practice Before Analysis

- Check for missing values (NA).
- Verify measurement units (e.g., cm, g, mg).
- Confirm category labels (e.g., Control, Stress).
- Check data consistency (outliers, duplicate entries, incorrect values).
- Understand the range and distribution of values.

7 The filter() Function

R & RStudio Fundamentals
From the History of R to Basic R Programming for Biological Data Science

The **filter()** function selects rows that satisfy a specified condition.
Research question: Which plants have a height greater than 40 cm?

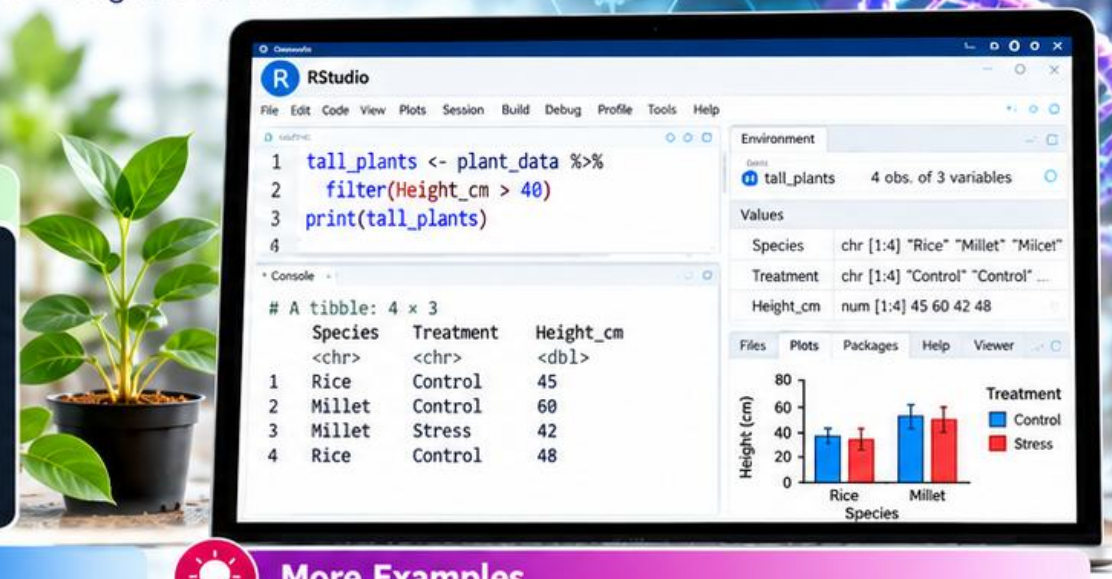
R Code

```
1  tall_plants <- plant_data %>%
2    filter(Height_cm > 40)
3
4  print(tall_plants)
```

Use the pipe %>% to pass the dataset to filter()

Output

```
> print(tall_plants)
# A tibble: 4 × 3
  Species   Treatment   Height_cm
  <chr>     <chr>       <dbl>
1 Rice      Control     45
2 Millet    Control     60
3 Millet    Stress      42
4 Rice      Control     48
```



Understanding the Result

- Only plants with **Height_cm > 40** are retained.
- Rows 1, 3, 4, and 5 from the original dataset meet the condition.
- The dataset now has 4 observations (rows) and the same 3 columns.
- **filter()** changes which observations are retained; it does not remove columns.

Other Useful Conditions

Condition	Meaning
==	Equal to
!=	Not equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
&	AND (both conditions must be TRUE)
|	OR (at least one condition is TRUE)
%in%	Match any value in a vector
!	NOT (negation)

More Examples

Research Question	R Code	Expected Result
Plants under Stress treatment	`plant_data %>% filter(Treatment == "Stress")`	Rows with Treatment = Stress
Millet plants only	`plant_data %>% filter(Species == "Millet")`	Rows for Millet
Height between 40 and 50 cm	`plant_data %>% filter(Height_cm >= 40 & Height_cm <= 50)`	Plants with height in 40–50 cm
Rice or height > 50 cm	`plant_data %>% filter(Species == "Rice" | Height_cm > 50)`	Rice plants or tall plants
Exclude Control	`plant_data %>% filter(Treatment != "Control")`	Non-control plants

Important Tips

- Use logical conditions to select biologically meaningful subsets.
- Combine multiple conditions using & (AND) or | (OR).
- Use **%in%** when filtering for multiple category levels (e.g., `filter(Species %in% c("Rice", "Millet"))`).
- Always inspect the output after filtering.
- Check for missing values (NA) if expected in real datasets.

Biological Applications

- Select plants with high biomass or specific traits.
- Filter samples under drought, salinity, or heat stress.
- Extract a subset of gene-expression data (e.g., DEGs).
- Select samples from a particular tissue or time point.
- Identify outliers or remove low-quality observations.

8 The select() Function

The **select()** function extracts specific columns from a dataset.

Research question: How can we retain only species identity and plant height?

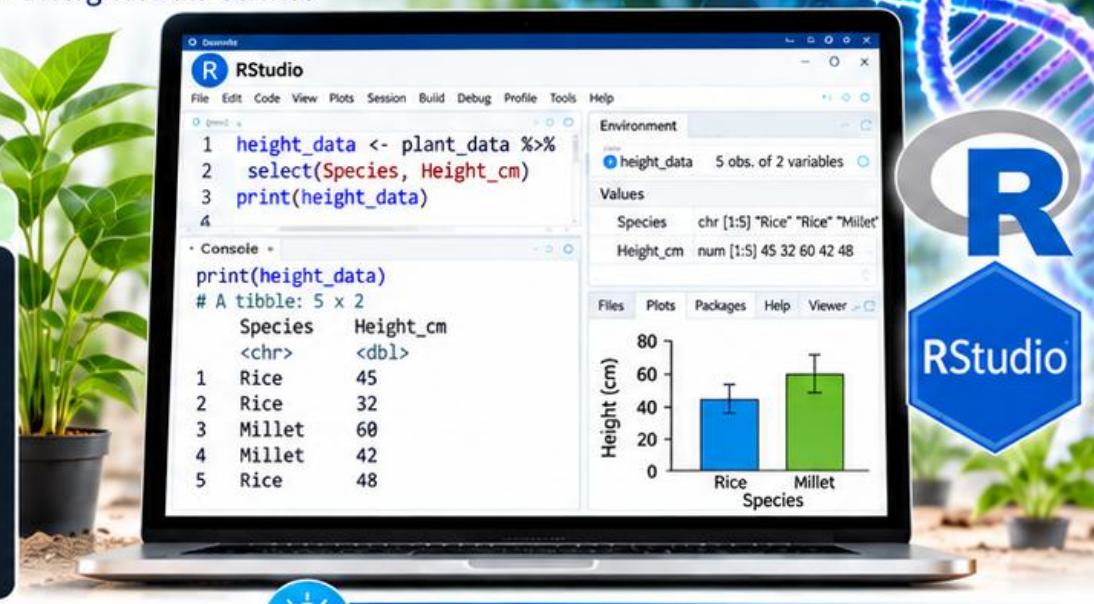
R Code

```
1  height_data <- plant_data %>%
2    select(Species, Height_cm)
3
   print(height_data)
```

Specify the column names to keep.

Output

```
>print(height_data)
# A tibble: 5 x 2
      Species    Height_cm
      <chr>      <dbl>
  1   Rice       45
  2   Rice       32
  3   Millet     60
  4   Millet     42
  5   Rice       48
```



More Examples

Purpose	R Code	Description
Select multiple columns	`plant_data %>% select(Species, Treatment, Height_cm)`	Keep specified columns.
Reorder columns	`plant_data %>% select(Height_cm, Species)`	Change column order.
Exclude columns	`plant_data %>% select(-Treatment)`	Remove a column.
Select columns by position	`plant_data %>% select(1, 3)`	Select 1st and 3rd columns.
Select columns by name pattern	`plant_data %>% select(starts_with("H"))`	Select columns starting with "H" (e.g., Height).

Useful Helper Functions

Function	Description
`starts_with("text")`	Select columns starting with a prefix.
`ends_with("text")`	Select columns ending with a suffix.
`contains("text")`	Select columns containing a string.
`matches("regex")`	Select columns using a regular expression.
`everything()`	Select all columns.
`-column_name`	Exclude a specific column.

Key Points

1. **select()** chooses which columns to keep; it does not change rows.
2. Column names are case-sensitive.
3. You can select, reorder, or exclude columns.
4. Helper functions (**starts_with**, **ends_with**, **contains**, **matches**) are useful for large datasets.
5. **select()** is often the first step in data cleaning and analysis.

Biological Applications

- Selecting gene IDs, chromosome coordinates, or expression values.
- Extracting specific biochemical parameters (e.g., chlorophyll, proline).
- Retaining sample identifiers and metadata.
- Working with large RNA-seq or metabolomics datasets with many columns.
- Preparing clean datasets for downstream analysis and visualisation.

9 The mutate() Function

R & RStudio Fundamentals
From the History of R to Basic R Programming
for Biological Data Science

The **mutate()** function creates new variables or modifies existing ones.
Research question: How can plant height be converted from centimetres to metres?

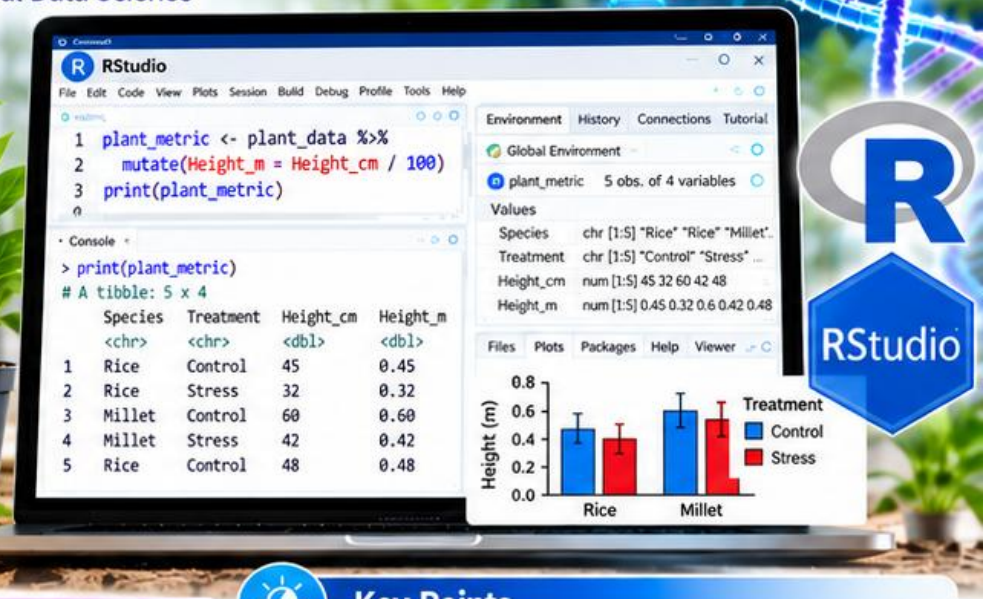
R Code

```
1  plant_metric <- plant_data %>%
2    mutate(Height_m = Height_cm / 100)
3
4  print(plant_metric)
```

Create a new variable Height_m by dividing Height_cm by 100.

Output

```
> print(plant_metric)
# A tibble: 5 x 4
  Species   Treatment  Height_cm  Height_m
  <chr>     <chr>      <dbl>      <dbl>
1 Rice      Control    45         0.45
2 Rice      Stress     32         0.32
3 Millet    Control    60         0.60
4 Millet    Stress     42         0.42
5 Rice      Control    48         0.48
```



More Examples of mutate()

Purpose	R Code	Description
Unit conversion (cm to m)	`plant_data %>% mutate(Height_m = Height_cm / 100)`	Convert height from centimetres to metres.
Create categorical variable	`plant_data %>% mutate(Size_class = if_else(Height_cm > 50, "Tall", "Short"))`	Classify plants based on height.
Log transformation	`plant_data %>% mutate(log_height = log(Height_cm))`	Apply log transformation (for skewed data).
Calculate ratio	`plant_data %>% mutate(Height_ratio = Height_cm / mean(Height_cm))`	Calculate height relative to the mean.
Create index	`plant_data %>% mutate(Stress_index = if_else(Treatment == "Stress", 1, 0))`	Create a binary indicator for stress treatment.
Modify existing variable	`plant_data %>% mutate(Height_cm = round(Height_cm, 1))`	Round height to one decimal place.

Common Useful Functions

Function	Use
`mutate()`	Create or modify variables.
`if_else()`	Conditional logic (preferred over `ifelse()`).
`case_when()`	Multiple conditions.
`lag()`	Use previous row value.
`lead()`	Use next row value.
`row_number()`	Add row numbers.
`across()`	Apply a function to multiple columns.
`scale()`	Standardise (mean = 0, sd = 1).
`log(),log2()`	Log transformation.
`round()`	Round numeric values.

Key Points

1. **mutate()** does not remove existing columns; it adds new variables or modifies them.
2. You can create multiple new variables in one **mutate()**.
3. Useful for unit conversion, categorical variables, log transformation, ratios, and indices.
4. Works with numerical, categorical, and logical operations.
5. Choose transformations based on the biological question and data distribution.

Biological Applications

- Convert measurement units (e.g., cm to m, g to kg).
- Calculate derived traits (e.g., biomass index, stress index).
- Normalise or transform data (e.g., log, z-score).
- Generate categorical classes (e.g., tall/short, tolerant/sensitive).
- Prepare variables for statistical analysis and visualisation.

10 The arrange() Function

The **arrange()** function sorts observations according to one or more variables.

Research question: Which plants are tallest?

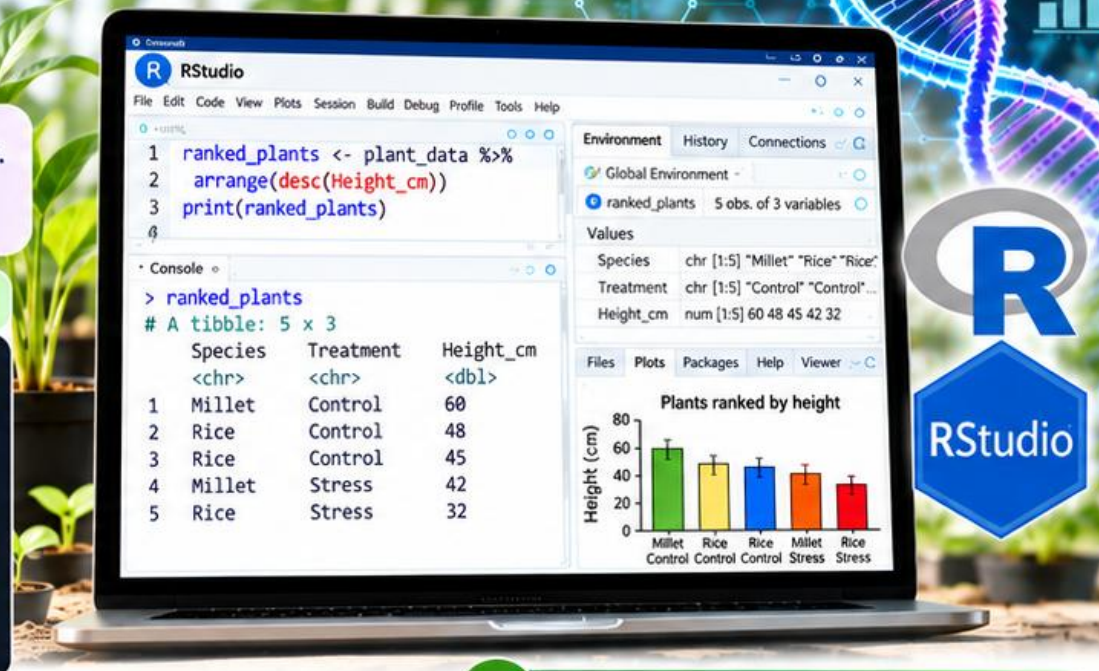
R Code

```
1  ranked_plants <- plant_data %>%
2    arrange(desc(Height_cm))
3
4  print(ranked_plants)
```

Use desc() for descending order (largest to smallest).

Output

```
> print(ranked_plants)
# A tibble: 5 x 3
  Species   Treatment   Height_cm
  <chr>     <chr>       <dbl>
1 Millet    Control     60
2 Rice      Control     48
3 Rice      Control     45
4 Millet    Stress      42
5 Rice      Stress      32
```



Different Ways to Use arrange()

Purpose	R Code	Description	Expected Result
Ascending order (smallest to largest)	`plant_data %>% arrange(Height_cm)`	Sort by height (low to high).	32, 42, 45, 48, 60
Descending order (largest to smallest)	`plant_data %>% arrange(desc(Height_cm))`	Sort by height (high to low).	60, 48, 45, 42, 32
Sort by multiple variables	`plant_data %>% arrange(Species, desc(Height_cm))`	First by Species (A–Z), then height (high to low).	Millet (60, 42), Rice (48, 45, 32)
Arrange by categorical variable	`plant_data %>% arrange(Treatment)`	Sort by treatment (Control, Stress).	Control rows first then Stress rows.
Custom order	`plant_data %>% mutate(Treatment = factor(Treatment, levels = c("Control", "Stress"))) %>% arrange(Treatment, desc(Height_cm))`	Define a custom order for categories and then sort.	Control (48, 45, 60) then Stress (42, 32) (order as specified)

Key Points

1. **arrange()** changes the order of rows; it does not remove or modify data.
2. By default, sorting is in ascending order.
3. Use **desc()** for descending order.
4. You can sort by multiple variables.
5. Works with both numerical and categorical variables.
6. Useful for ranking and prioritising observations.
7. Combine with other functions (e.g., **group_by()**, **mutate()**, **select()**) in a pipeline.

Biological Applications

- Rank genotypes by yield, biomass, or stress tolerance.
- Sort genes by expression level (e.g., log2FC, TPM).
- Identify top/bottom performing treatments or samples.
- Arrange samples by time points, developmental stages, or phenotypes.
- Prioritise candidate genes or markers based on statistics (e.g., p-value, fold change).
- Useful for data summarisation, visualisation, and reporting in biological research.

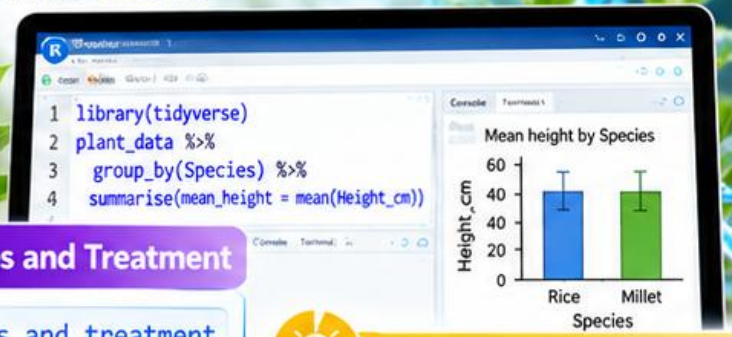
Tips and Best Practices

- Check variable type before sorting (numeric or factor).
- For categorical data, consider setting factor levels.
- Use multiple variables for meaningful ordering.
- Verify the sorted output using head() or print().
- Combine with filter() or select() for targeted analysis.

11 The group_by() Function

R & RStudio Fundamentals
From the History of R to Basic R Programming for Biological Data Science

The **group_by()** function defines groups of observations for subsequent calculations.



1 Group by Species

```
1  # Group by species
2  plant_data %>%
3    group_by(Species)
```

Output (grouped tibble)

```
# A tibble: 5 x 3
# Groups:   Species [2]
  Species   Treatment   Height_cm
  <chr>     <chr>           <dbl>
1 Rice      Control            45
2 Rice      Stress             32
3 Millet    Control            60
4 Millet    Stress             42
5 Rice      Control            48
```

What it does?

- Creates groups based on **Species**. Subsequent operations (e.g., summarise, mutate) will be performed within each species.

2 Group by Treatment

```
1  # Group by treatment
2  plant_data %>%
3    group_by(Treatment)
```

Output (grouped tibble)

```
# A tibble: 5 x 3
# Groups:   Treatment [2]
  Species   Treatment   Height_cm
  <chr>     <chr>           <dbl>
1 Rice      Control            45
2 Rice      Stress             32
3 Millet    Control            60
4 Millet    Stress             42
5 Rice      Control            48
```

What it does?

- Creates groups based on **Treatment**. Allows treatment-wise summaries and comparisons.

3 Group by Species and Treatment

```
1  # Group by species and treatment
2  plant_data %>%
3    group_by(Species, Treatment)
```

Output (grouped tibble)

```
# A tibble: 5 x 3
# Groups:   Species, Treatment [4]
  Species   Treatment   Height_cm
  <chr>     <chr>           <dbl>
1 Rice      Control            45
2 Rice      Stress             32
3 Millet    Control            60
4 Millet    Stress             42
5 Rice      Control            48
```

What it does?

- Creates groups based on both variables. Enables comparisons within each species × treatment combination.

Key Points

1. **group_by()** does not calculate any summary by itself; it only sets the grouping structure.
2. Use multiple variables with commas: group_by(var1, var2).
3. Works with other functions like summarise(), mutate(), filter(), and arrange().
4. To remove grouping, use ungroup()

Biological Applications

- Species-wise or treatment-wise summary of plant height, yield, or biochemical parameters.
- Grouping gene expression by tissue, condition, or genotype.
- Comparing phenotypic or physiological traits across different groups.
- Useful for multi-factor analysis in transcriptomics, metabolomics and field experiments.

12 The summarise() Function

R & RStudio Fundamentals
From the History of R to Basic R Programming for Biological Data Science

The **summarise()** function reduces observations to summary values, often one row per group.
Research question: What are the mean height, standard deviation, and sample size for each species?

R Code

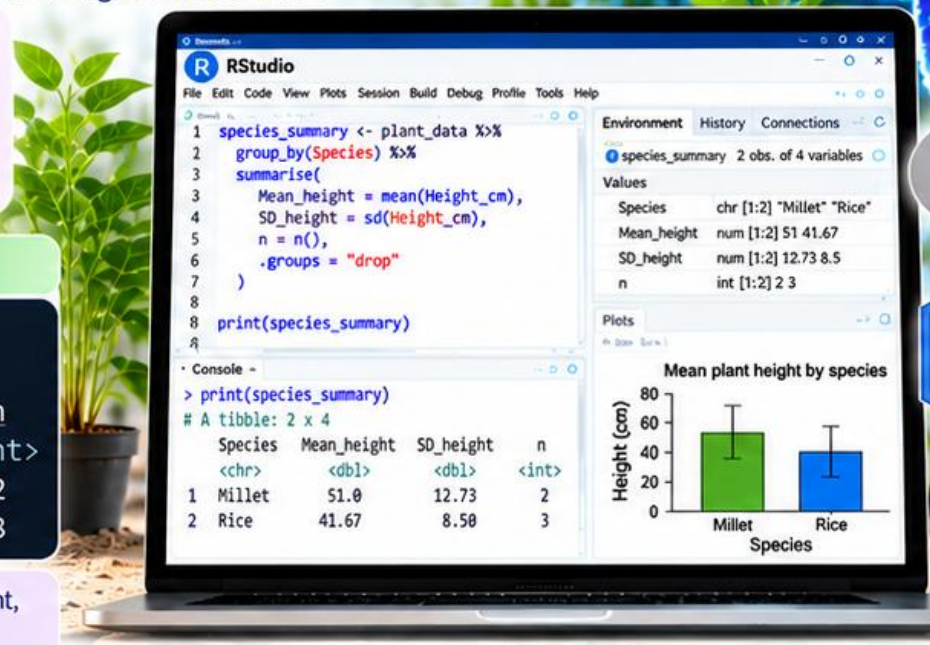
```r
species_summary <- plant_data %>%
  group_by(Species) %>%
  summarise(
    Mean_height = mean(Height_cm),
    SD_height = sd(Height_cm),
    n = n(),
    .groups = "drop"
  )

print(species_summary)
```

Output

```
> print(species_summary)
# A tibble: 2 x 4
  Species   Mean_height  SD_height     n
  <chr>       <dbl>>       <dbl>    <int>
1 Millet        51.0        12.73     2
2 Rice          41.67        8.50     3
```

One row per species summarising the mean plant height, standard deviation, and sample size.



More Examples of summarise()

Purpose	R Code	Description	Example Output (abridged)
Mean and SD by treatment	`plant_data %>% group_by(Treatment) %>% summarise(Mean = mean(Height_cm), SD = sd(Height_cm), n= n())`	Summary statistics for each treatment.	Treatment | Mean | SD | n Control | 51.0 | 7.55 | 3 Stress | 37.0 | 5.29 | 2
Mean by species and treatment	`plant_data %>% group_by(Species, Treatment) %>% summarise(Mean = mean(Height_cm), SD = sd(Height_cm), n = n())`	Summary for each species × treatment combination.	Species | Treatment | Mean | SD | n Millet | Control | 60 | NA | 1 Millet | Stress | 42 | NA | 1 Rice | Control | 46.5 | 2.12 | 2 Rice | Stress | 32 | NA | 1
Multiple statistics	`plant_data %>% group_by(Species) %>% summarise(Mean = mean(Height_cm), Median = median(Height_cm), Min = min(Height_cm), Max = max(Height_cm), n = n())`	Calculate several descriptive statistics in one step.	Species | Mean | Median | Min | Max | n Millet | 51.0 | 51.0 | 42 | 60 | 2 Rice | 41.67 | 45.0 | 32 | 48 | 3

Key Points

1. **summarise()** reduces multiple observations to summary values (often one row per group).
2. Common functions: **mean()**, **sd()**, **n()**, **median()**, **min()**, **max()**, **sum()**.
3. Use **group_by()** before summarise() to calculate group-wise summaries.
4. **.groups = "drop"** removes the grouping structure from tiiy result.
5. The result is a new data frame with summary statistics.
6. These are descriptive statistics, not statistical tests.

Biological Applications

- Calculate mean trait values (e.g., height, biomass, chlorophyll content) for each genotype or treatment.
- Summarise gene expression levels across replicates.
- Obtain summary statistics for different tissues, time points, or environmental conditions.
- Generate descriptive statistics for metabolomics, proteomics or other omics data.
- Useful for reporting results in tables and figures for scientific publications.

Important Considerations

- Check adequate biological replication (n).
- Consider data distribution and outliers.
- Use appropriate statistics based on the research question.
- summarise() provides descriptive statistics; inferential analysis requires additional tests (e.g., t-test, ANOVA).

13 Pipes: Combining Operations

R & RStudio Fundamentals
From the History of R to Basic R Programming for Biological Data Science

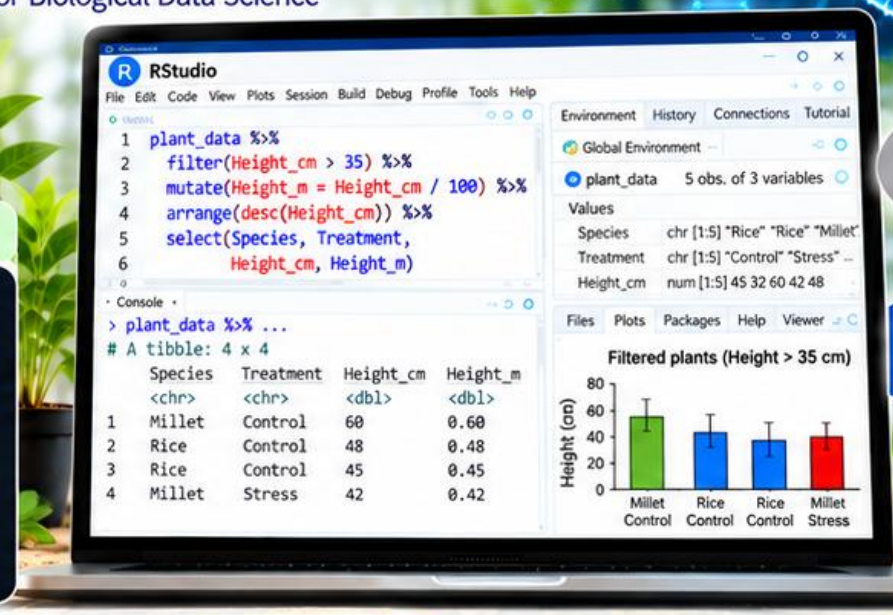
The **pipe operator %>%** passes the result of one operation to the next, helping create readable workflows.

R Code (Using %>%)

```
plant_data %>%
  filter(Height_cm > 35) %>%
  mutate(Height_m = Height_cm / 100) %>%
  arrange(desc(Height_cm)) %>%
  select(
    Species, Treatment,
  ) Height_cm, Height_m
```

Output

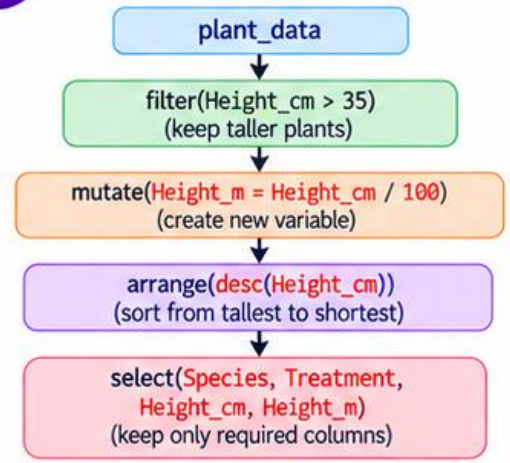
```
> plant_data %>% ...
# A tibble: 4 x 4
  Species  Treatment  Height_cm  Height_m
  <chr>    <chr>      <dbl>      <dbl>
1 Millet   Control    60         0.60
2 Rice     Control    48         0.48
3 Rice     Control    45         0.45
4 Millet   Stress     42         0.42
```



How the Workflow Works

1. Start with plant_data.
2. Retain observations with Height_cm > 35 using filter().
3. Create a new variable Height_m using mutate().
4. Sort the data from tallest to shortest using arrange(desc()).
5. Keep only the required columns using select().

Step-by-Step Illustration



Also Using the Native Pipe |>

R (≥ 4.1.0) also provides a native pipe operator |> with similar functionality.

```
plant_data |>
  filter(Height_cm > 35) |>
  mutate(Height_m = Height_cm / 100) |>
  arrange(desc(Height_cm)) |>
  select(Species, Treatment,
         Height_cm, Height_m)
```

Both %>% (from magrittr, loaded with tidyverse) and |> work similarly. %>% is widely used in tidyverse and older R versions, while |> is part of base R.

Key Advantages and Use Cases

- Improves readability and makes the workflow easy to follow.
- Allows chaining multiple operations in a logical order.
- Reduces the need for intermediate objects.
- Easier to reproduce and share analysis steps.
- Useful for processing large biological datasets (e.g., gene expression, phenotypic data, environmental data).

Best Practices

- Use meaningful variable names.
- Keep each step simple and well-commented.
- Verify intermediate results (e.g., using head(), glimpse()).
- Combine pipes with other tidyverse functions for robust analysis.

14 Integrated Example: Mean Height by Species and Treatment

R & RStudio Fundamentals
From the History of R to Basic R Programming for Biological Data Science

Research question: What is the mean plant height for each species–treatment combination?

R Code (Using tidyverse)

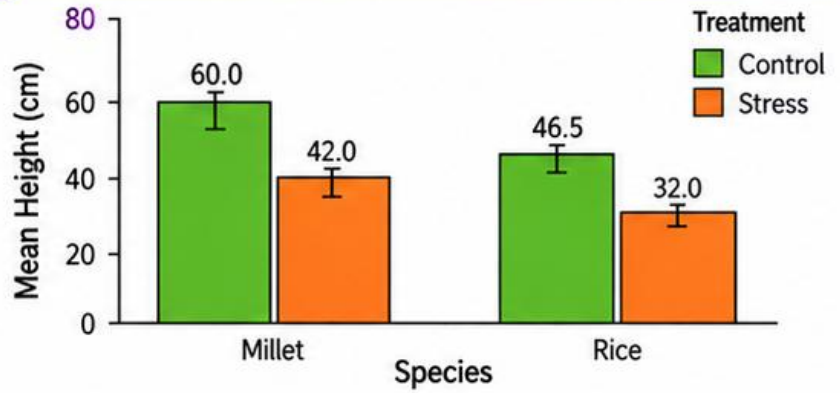
```r
treatment_summary <- plant_data %>%
  group_by(Species, Treatment) %>%
  summarise(
    Mean_height = mean(Height_cm),
    n = n(),
    .groups = "drop"
  ) %>%
  arrange(Species, Treatment)

print(treatment_summary)
```

Output (Tibble)

```
> print(treatment_summary)
# A tibble: 4 x 4
  Species  Treatment  Mean_height     n
  <chr>    <chr>        <dbl>     <int>
1 Millet   Control       60.0       1
2 Millet   Stress        42.0       1
3 Rice     Control       46.5       2
4 Rice     Stress        32.0       1
```

Visualization: Mean Height by Species and Treatment



Step-by-Step Workflow

1. Start with plant_data.
2. Group the data by Species and Treatment using group_by().
3. Calculate the mean height and sample size using summarise().
4. Remove grouping with .groups = "drop".
5. Sort the result using arrange().
6. View the final summary table and visualise the results (optional).

Interpretation

- In this illustrative dataset, plant height is lower under Stress compared to Control within both species.
- Millet shows a larger reduction (60.0 → 42.0 cm) than Rice (46.5 → 32.0 cm).
- However, the dataset is small and unbalanced, so we cannot establish a statistically significant treatment effect.
- This example demonstrates data manipulation and summarisation using tidyverse, not a formal statistical test.

Key Points

- Combines multiple tidyverse functions: group_by(), summarise(), arrange().
- Produces one row per species–treatment combination.
- Useful for calculating descriptive statistics (mean, SD, n) for biological experiments.
- Can be extended to include other statistics (e.g., sd(), median()) or additional variables.
- Visualization helps to quickly compare patterns across groups.

Biological Applications

- Compare growth, yield, or biochemical parameters across species and treatments.
- Summarise gene expression levels by condition, tissue, or genotype.
- Analyse physiological or biochemical responses to stress (e.g., drought, salinity, nutrient deficiency).
- Generate summary tables for reports, figures, and publications.
- Scalable to large datasets with many species, treatments, or time points.